第10章 内 部 类

可以将一个类的定义放在另一个类的定义内部，这就是内部类。

内部类是一种非常有用的特性，因为它允许你把一些逻辑相关的类组织在一起，并控制位于内部的类的可视性。然而必须要了解，内部类与组合是完全不同的概念，这一点很重要。

在最初，内部类看起来就像是一种代码隐藏机制：将类置于其他类的内部。但是，你将会了解到，内部类远不止如此，它了解外围类，并能与之通信；而且你用内部类写出的代码更加优雅而清晰，尽管并不总是这样。

最初，内部类可能看起来有些奇怪，而且要花些时间才能在设计中轻松地使用它们。对内部类的需求并非总是很明显的，但是在描述完内部类的基本语法与语义之后，10.8节就应该使得内部类的益处明确显现了。

在10.8节之后，本章剩余部分包含了对内部类语法更加详尽的探索，这些特性是为了语言的完备性而设计的，但是你也许不需要使用它们，至少一开始不需要。因此，本章最初的部分也许就是你现在所需的全部，你可以将更详尽的探索当作参考资料。

10.1 创建内部类

创建内部类的方式就如同你想的一样——把类的定义置于外围类的里面：

```
//: innerclasses/Parcel1.java
// Creating inner classes.

public class Parcel1 {
  class Contents {
    private int i = 11;
    public int value() { return i; }
  }
  class Destination {
    private String label;
    Destination(String whereTo) {
      label = whereTo;
    }
    String readLabel() { return label; }
  }
  // Using inner classes looks just like
  // using any other class, within Parcel1:
  public void ship(String dest) {
    Contents c = new Contents();
    Destination d = new Destination(dest);
    System.out.println(d.readLabel());
  }
  public static void main(String[] args) {
    Parcel1 p = new Parcel1();
    p.ship("Tasmania");
  }
} /* Output:
Tasmania
*///:~
```


当我们在**ship()**方法里面使用内部类的时候，与使用普通类没什么不同。在这里，实际的区别只是内部类的名字是嵌套在**Parcel1**里面的。不过你将会看到，这并不是唯一的区别。

更典型的情况是，外部类将有一个方法，该方法返回一个指向内部类的引用，就像在**to()**和

contents()方法中看到的那样：

```
//: innerclasses/Parcel2.java
// Returning a reference to an inner class.

public class Parcel2 {
  class Contents {
    private int i = 11;
    public int value() { return i; }
  }
  class Destination {
    private String label;
    Destination(String whereTo) {
      label = whereTo;
    }
    String readLabel() { return label; }
  }
  public Destination to(String s) {
    return new Destination(s);
  }
  public Contents contents() {
    return new Contents();
  }
  public void ship(String dest) {
    Contents c = contents();
    Destination d = to(dest);
    System.out.println(d.readLabel());
  }
  public static void main(String[] args) {
    Parcel2 p = new Parcel2();
    p.ship("Tasmania");
    Parcel2 q = new Parcel2();
    // Defining references to inner classes:
    Parcel2.Contents c = q.contents();
    Parcel2.Destination d = q.to("Borneo");
  }
} /* Output:
Tasmania
*///:~
```

346

如果想从外部类的非静态方法之外的任意位置创建某个内部类的对象，那么必须像在**main()**方法中那样，具体地指明这个对象的类型：**OuterClassName.InnerClassName**。

练习1：(1) 编写一个名为**Outer**的类，它包含一个名为**Inner**的类。在**Outer**中添加一个方法，它返回一个**Inner**类型的对象。在**main()**中，创建并初始化一个指向某个**Inner**对象的引用。

10.2 链接到外部类

到目前为止，内部类似乎还只是一种名字隐藏和组织代码的模式。这些是很有用，但还不是最引人注目的，它还有其他的用途。当生成一个内部类的对象时，此对象与制造它的外围对象（enclosing object）之间就有了一种联系，所以它能访问其外围对象的所有成员，而不需要任何特殊条件。此外，内部类还拥有其外围类的所有元素的访问权[⊖]。下面的例子说明了这点：

347

```
//: innerclasses/Sequence.java
// Holds a sequence of Objects.

interface Selector {
  boolean end();
  Object current();
  void next();
}
```

⊖ 这与C++嵌套类的设计非常不同，在C++中只是单纯的名字隐藏机制，与外围对象没有联系，也没有隐含的访问权。

```
public class Sequence {
  private Object[] items;
  private int next = 0;
  public Sequence(int size) { items = new Object[size]; }
  public void add(Object x) {
    if(next < items.length)
      items[next++] = x;
  }
  private class SequenceSelector implements Selector {
    private int i = 0;
    public boolean end() { return i == items.length; }
    public Object current() { return items[i]; }
    public void next() { if(i < items.length) i++; }
  }
  public Selector selector() {
    return new SequenceSelector();
  }
  public static void main(String[] args) {
    Sequence sequence = new Sequence(10);
    for(int i = 0; i < 10; i++)
      sequence.add(Integer.toString(i));
    Selector selector = sequence.selector();
    while(!selector.end()) {
      System.out.print(selector.current() + " ");
      selector.next();
    }
  }
} /* Output:
0 1 2 3 4 5 6 7 8 9
*///:~
```

348

Sequence类只是一个固定大小的**Object**的数组，以类的形式包装了起来。可以调用**add()**在序列末增加新的**Object**（只要还有空间）。要获取**Sequence**中的每一个对象，可以使用**Selector**接口。这是“迭代器”设计模式的一个例子，在本书稍后的部分将更多地学习它。**Selector**允许你检查序列是否到末尾了（**end()**），访问当前对象（**current()**），以及移到序列中的下一个对象（**next()**）。因为**Selector**是一个接口，所以别的类可以按它们自己的方式来实现这个接口，并且另的方法能以此接口为参数，来生成更加通用的代码。

这里，**SequenceSelector**是提供**Selector**功能的**private**类。可以看到，在**main()**中创建了一个**Sequence**，并向其中添加了一些**String**对象。然后通过调用**selector()**获取一个**Selector**，并用它在**Sequence**中移动和选择每一个元素。

最初看到**SequenceSelector**，可能会觉得它只不过是另一个内部类罢了。但请仔细观察它，注意方法**end()**、**current()**和**next()**都用到了**objects**，这是一个引用，它并不是**SequenceSelector**的一部分，而是外围类中的一个**private**字段。然而内部类可以访问其外围类的方法和字段，就像自己拥有它们似的，这带来了很大的方便，就如前面的例子所示。

所以内部类自动拥有对其外围类所有成员的访问权。这是如何做到的呢？当某个外围类的对象创建了一个内部类对象时，此内部类对象必定会秘密地捕获一个指向那个外围类对象的引用。然后，在你访问此外围类的成员时，就是用那个引用来选择外围类的成员。幸运的是，编译器会帮你处理所有的细节，但你现在可以看到：内部类的对象只能在与其外围类的对象相关联的情况下才能被创建（就像你应该看到的，在内部类是非**static**类时）。构建内部类对象时，需要一个指向其外围类对象的引用，如果编译器访问不到这个引用就会报错。不过绝大多数时候这都无需程序员操心。

349

练习2：(1) 创建一个类，它持有一个**String**，并且有一个显示这个**String**的**toString()**方法。将你的新类的若干个对象添加到一个**Sequence**对象中，然后显示它们。

练习3：(1) 修改练习1，使得**Outer**类包含一个**private String**域（由构造器初始化），而

Inner包含一个显示这个域的**toString()**方法。创建一个**Inner**类型的对象并显示它。

10.3 使用.this与.new

如果你需要生成对外部类对象的引用，可以使用外部类的名字后面紧跟圆点和**this**。这样产生的引用自动地具有正确的类型，这一点在编译期就被知晓并受到检查，因此没有任何运行时开销。下面的示例展示了如何使用**.this**：

```
//: innerclasses/DotThis.java
// Qualifying access to the outer-class object.

public class DotThis {
  void f() { System.out.println("DotThis.f()"); }
  public class Inner {
    public DotThis outer() {
      return DotThis.this;
      // A plain "this" would be Inner's "this"
    }
  }
  public Inner inner() { return new Inner(); }
  public static void main(String[] args) {
    DotThis dt = new DotThis();
    DotThis.Inner dti = dt.inner();
    dti.outer().f();
  }
} /* Output:
DotThis.f()
*///:~
```

有时你可能想要告知某些其他对象，去创建其某个内部类的对象。要实现此目的，你必须在**new**表达式中提供对其他外部类对象的引用，这是需要使用**.new**语法，就像下面这样：

```
//: innerclasses/DotNew.java
// Creating an inner class directly using the .new syntax.
public class DotNew {
  public class Inner {}
  public static void main(String[] args) {
    DotNew dn = new DotNew();
    DotNew.Inner dni = dn.new Inner();
  }
} ///:~
```

350

要想直接创建内部类的对象，你不能按照你想象的方式，去引用外部类的名字**DotNew**，而是必须使用外部类的对象来创建该内部类对象，就像在上面的程序中所看到的那样。这也解决了内部类名字作用域的问题，因此你不必声明（实际上你不能声明）**dn.new DotNew.Inner()**。

在拥有外部类对象之前是不可能创建内部类对象的。这是因为内部类对象会暗暗地连接到创建它的外部类对象上。但是，如果你创建的是*嵌套类*（静态内部类），那么它就不需要对外部类对象的引用。

下面你可以看到将**.new**应用于**Parcel**的示例：

```
//: innerclasses/Parcel3.java
// Using .new to create instances of inner classes.

public class Parcel3 {
  class Contents {
    private int i = 11;
    public int value() { return i; }
  }
  class Destination {
    private String label;
    Destination(String whereTo) { label = whereTo; }
```

```
    String readLabel() { return label; }
  }
  public static void main(String[] args) {
    Parcel3 p = new Parcel3();
    // Must use instance of outer class
    // to create an instance of the inner class:
    Parcel3.Contents c = p.new Contents();
    Parcel3.Destination d = p.new Destination("Tasmania");
  }
} ///:~
```

351

练习4：(2) 在**Sequence.SequenceSelector**类中增加一个方法，它可以生成对外部类Sequence的引用。

练习5：(1) 创建一个包含内部类的类，在另一个独立的类中，创建此内部类的实例。

10.4 内部类与向上转型

当将内部类向上转型为其基类，尤其是转型为一个接口的时候，内部类就有了用武之地。(从实现了某个接口的对象，得到对此接口的引用，与向上转型为这个对象的基类，实质上效果是一样的。) 这是因为此内部类——某个接口的实现——能够完全不可见，并且不可用。所得到的只是指向基类或接口的引用，所以能够很方便地隐藏实现细节。

我们可以创建前一个示例的接口：

```
//: innerclasses/Destination.java
public interface Destination {
  String readLabel();
} ///:~
```

```
//: innerclasses/Contents.java
public interface Contents {
  int value();
} ///:~
```

现在**Contents**和**Destination**表示客户端程序员可用的接口。(记住，接口的所有成员自动被设置为**public**的。)

当取得了一个指向基类或接口的引用时，甚至可能无法找出它确切的类型，看下面的例子：

```
//: innerclasses/TestParcel.java

class Parcel4 {
  private class PContents implements Contents {
    private int i = 11;
    public int value() { return i; }
  }
  protected class PDestination implements Destination {
    private String label;
    private PDestination(String whereTo) {
      label = whereTo;
    }
    public String readLabel() { return label; }
  }
  public Destination destination(String s) {
    return new PDestination(s);
  }
  public Contents contents() {
    return new PContents();
  }
}

public class TestParcel {
  public static void main(String[] args) {
    Parcel4 p = new Parcel4();
    Contents c = p.contents();
```

352

```
    Destination d = p.destination("Tasmania");
    // Illegal -- can't access private class:
    //! Parcel4.PContents pc = p.new PContents();
  }
} ///:~
```

Parcel4中增加了一些新东西：内部类**PContents**是**private**，所以除了**Parcel4**，没有人能访问它。**PDestination**是**protected**，所以只有**Parcel4**及其子类、还有与**Parcel4**同一个包中的类（因为**protected**也给予了包访问权）能访问**PDestination**，其他类都不能访问**PDestination**。这意味着，如果客户端程序员想了解或访问这些成员，那是要受到限制的。实际上，甚至不能向下转型成**private**内部类（或**protected**内部类，除非是继承自它的子类），因为不能访问其名字，就像在**TestParcel**类中看到的那样。于是，**private**内部类给类的设计者提供了一种途径，通过这种方式可以完全阻止任何依赖于类型的编码，并且完全隐藏了实现的细节。此外，从客户端程序员的角度来看，由于不能访问任何新增加的、原本不属于公共接口的方法，所以扩展接口是没有价值的。这也给Java编译器提供了生成更高效代码的机会。

练习6：(2) 在第一个包中创建一个至少有一个方法的接口。然后在第二个包内创建一个类，在其中增加一个**protected**的内部类以实现那个接口。在第三个包中，继承这个类，并在一个方法中返回该**protected**内部类的对象，在返回的时候向上转型为第一个包中的接口的类型。 353

练习7：(2) 创建一个含有**private**域和**private**方法的类。创建一个内部类，它有一个方法可用来修改外围类的域，并调用外围类的方法。在外围类的另一方法中，创建此内部类的对象，并且调用它的方法，然后说明对外围类对象的影响。

练习8：(2) 确定外部类是否可以访问其内部类的**private**元素。

10.5 在方法和作用域内的内部类

到目前为止，读者所看到的只是内部类的典型用途。通常，如果所读、写的代码包含了内部类，那么它们都是“平凡的”内部类，简单并且容易理解。然而，内部类的语法覆盖了大量其他的更加难以理解的技术。例如，可以在一个方法里面或者在任意的作用域内定义内部类。这么做有两个理由：

1) 如前所示，你实现了某类型的接口，于是可以创建并返回对其的引用。

2) 你要解决一个复杂的问题，想创建一个类来辅助你的解决方案，但是又不希望这个类是公共可用的。

在后面的例子中，先前的代码将被修改，以用来实现：

1) 一个定义在方法中的类。

2) 一个定义在作用域内的类，此作用域在方法的内部。

3) 一个实现了接口的匿名类。

4) 一个匿名类，它扩展了有非默认构造器的类。

5) 一个匿名类，它执行字段初始化。 354

6) 一个匿名类，它通过实例初始化实现构造（匿名类不可能有构造器）。

第一个例子展示了在方法的作用域内（而不是在其他类的作用域内）创建一个完整的类。这被称作*局部内部类*：

```
//: innerclasses/Parcel5.java
// Nesting a class within a method.

public class Parcel5 {
  public Destination destination(String s) {
    class PDestination implements Destination {
      private String label;
```

```
      private PDestination(String whereTo) {
        label = whereTo;
      }
      public String readLabel() { return label; }
    }
    return new PDestination(s);
  }
  public static void main(String[] args) {
    Parcel5 p = new Parcel5();
    Destination d = p.destination("Tasmania");
  }
} ///:~
```

PDestination类是**destination()**方法的一部分，而不是**Parcel5**的一部分。所以，在**destination()**之外不能访问**PDestination**。注意出现在**return**语句中的向上转型——返回的是**Destination**的引用，它是**PDestination**的基类。当然，在**destination()**中定义了内部类**PDestination**，并不意味着一旦**dest()**方法执行完毕，**PDestination**就不可用了。

你可以在同一个子目录下的任意类中对某个内部类使用类标识符**PDestination**，这并不会有命名冲突。

下面的例子展示了如何在任意的作用域内嵌入一个内部类：

355

```
//: innerclasses/Parcel6.java
// Nesting a class within a scope.
public class Parcel6 {
  private void internalTracking(boolean b) {
    if(b) {
      class TrackingSlip {
        private String id;
        TrackingSlip(String s) {
          id = s;
        }
        String getSlip() { return id; }
      }
      TrackingSlip ts = new TrackingSlip("slip");
      String s = ts.getSlip();
    }
    // Can't use it here! Out of scope:
    //! TrackingSlip ts = new TrackingSlip("x");
  }
  public void track() { internalTracking(true); }
  public static void main(String[] args) {
    Parcel6 p = new Parcel6();
    p.track();
  }
} ///:~
```

TrackingSlip类被嵌入在**if**语句的作用域内，这并不是说该类的创建是有条件的，它其实与别的类一起编译过了。然而，在定义**TrackingSlip**的作用域之外，它是不可用的；除此之外，它与普通的类一样。

练习9：(1) 创建一个至少有一个方法的接口。在某个方法内定义一个内部类以实现此接口，这个方法返回对此接口的引用。

练习10：(1) 重复前一个练习，但将内部类定义在某个方法的一个作用域内。

练习11：(2) 创建一个**private**内部类，让它实现一个**public**接口。写一个方法，它返回一个指向此**private**内部类的实例的引用，并将此引用向上转型为该接口类型。通过尝试向下转型，说明此内部类被完全隐藏了。

10.6 匿名内部类

下面的例子看起来有点奇怪：

356

```
//: innerclasses/Parcel7.java
// Returning an instance of an anonymous inner class.

public class Parcel7 {
  public Contents contents() {
    return new Contents() { // Insert a class definition
      private int i = 11;
      public int value() { return i; }
    }; // Semicolon required in this case
  }
  public static void main(String[] args) {
    Parcel7 p = new Parcel7();
    Contents c = p.contents();
  }
} ///:~
```

contents()方法将返回值的生成与表示这个返回值的类的定义结合在一起！另外，这个类是匿名的，它没有名字。更糟的是，看起来似乎是你正要创建一个**Contents**对象。但是然后（在到达语句结束的分号之前）你却说："等一等，我想在这里插入一个类的定义。"

这种奇怪的语法指的是："创建一个继承自**Contents**的匿名类的对象。"通过**new**表达式返回的引用被自动向上转型为对**Contents**的引用。上述匿名内部类的语法是下述形式的简化形式：

```
//: innerclasses/Parcel7b.java
// Expanded version of Parcel7.java

public class Parcel7b {
  class MyContents implements Contents {
    private int i = 11;
    public int value() { return i; }
  }
  public Contents contents() { return new MyContents(); }
  public static void main(String[] args) {
    Parcel7b p = new Parcel7b();
    Contents c = p.contents();
  }
} ///:~
```

在这个匿名内部类中，使用了默认的构造器来生成**Contents**。下面的代码展示的是，如果你的基类需要一个有参数的构造器，应该怎么办： 357

```
//: innerclasses/Parcel8.java
// Calling the base-class constructor.

public class Parcel8 {
  public Wrapping wrapping(int x) {
    // Base constructor call:
    return new Wrapping(x) { // Pass constructor argument.
      public int value() {
        return super.value() * 47;
      }
    }; // Semicolon required
  }
  public static void main(String[] args) {
    Parcel8 p = new Parcel8();
    Wrapping w = p.wrapping(10);
  }
} ///:~
```

只需简单地传递合适的参数给基类的构造器即可，这里是将**x**传进**new Wrapping(x)**。尽管**Wrapping**只是一个具有具体实现的普通类，但它还是被其导出类当作公共"接口"来使用：

```
//: innerclasses/Wrapping.java
public class Wrapping {
  private int i;
  public Wrapping(int x) { i = x; }
```

```
  public int value() { return i; }
} ///:~
```

你会注意到，**Wrapping**拥有一个要求传递一个参数的构造器，这使得事情变得更加有趣了。

在匿名内部类末尾的分号，并不是用来标记此内部类结束的。实际上，它标记的是表达式的结束，只不过这个表达式正巧包含了匿名内部类罢了。因此，这与别的地方使用的分号是一致的。

在匿名类中定义字段时，还能够对其执行初始化操作：

358

```
//: innerclasses/Parcel9.java
// An anonymous inner class that performs
// initialization. A briefer version of Parcel5.java.

public class Parcel9 {
  // Argument must be final to use inside
  // anonymous inner class:
  public Destination destination(final String dest) {
    return new Destination() {
      private String label = dest;
      public String readLabel() { return label; }
    };
  }
  public static void main(String[] args) {
    Parcel9 p = new Parcel9();
    Destination d = p.destination("Tasmania");
  }
} ///:~
```

如果定义一个匿名内部类，并且希望它使用一个在其外部定义的对象，那么编译器会要求其参数引用是**final**的，就像你在**destination()**的参数中看到的那样。如果你忘记了，将会得到一个编译时错误消息。

如果只是简单地给一个字段赋值，那么此例中的方法是很好的。但是，如果想做一些类似构造器的行为，该怎么办呢？在匿名类中不可能有命名构造器（因为它根本没名字！），但通过实例初始化，就能够达到为匿名内部类创建一个构造器的效果，就像这样：

```
//: innerclasses/AnonymousConstructor.java
// Creating a constructor for an anonymous inner class.
import static net.mindview.util.Print.*;

abstract class Base {
  public Base(int i) {
    print("Base constructor, i = " + i);
  }
  public abstract void f();
}

public class AnonymousConstructor {
  public static Base getBase(int i) {
    return new Base(i) {
      { print("Inside instance initializer"); }
      public void f() {
        print("In anonymous f()");
      }
    };
  }
  public static void main(String[] args) {
    Base base = getBase(47);
    base.f();
  }
} /* Output:
Base constructor, i = 47
Inside instance initializer
```

359

```
In anonymous f()
*///:~
```

在此例中，不要求变量i一定是final的。因为i被传递给匿名类的基类的构造器，它并不会在匿名类内部被直接使用。

下例是带实例初始化的“parcel”形式。注意destination()的参数必须是final的，因为它们是在匿名类内部使用的。

```
//: innerclasses/Parcel10.java
// Using "instance initialization" to perform
// construction on an anonymous inner class.

public class Parcel10 {
  public Destination
  destination(final String dest, final float price) {
    return new Destination() {
      private int cost;
      // Instance initialization for each object:
      {
        cost = Math.round(price);
        if(cost > 100)
          System.out.println("Over budget!");
      }
      private String label = dest;
      public String readLabel() { return label; }
    };
  }
  public static void main(String[] args) {
    Parcel10 p = new Parcel10();
    Destination d = p.destination("Tasmania", 101.395F);
  }
} /* Output:
Over budget!
*///:~
```

在实例初始化操作的内部，可以看到有一段代码，它们不能作为字段初始化动作的一部分来执行（就是if语句）。所以对于匿名类而言，实例初始化的实际效果就是构造器。当然它受到了限制——你不能重载实例初始化方法，所以你仅有一个这样的构造器。

匿名内部类与正规的继承相比有些受限，因为匿名内部类既可以扩展类，也可以实现接口，但是不能两者兼备。而且如果是实现接口，也只能实现一个接口。

练习12：(1) 重复练习7，这次使用匿名内部类。

练习13：(1) 重复练习9，这次使用匿名内部类。

练习14：(1) 修改interfaces/HorrorShow.java，用匿名类实现DangerousMonster和Vampire。

练习15：(2) 创建一个类，它有非默认的构造器（即需要参数的构造器），并且没有默认构造器（没有无参数的构造器）。创建第二个类，它包含一个方法，能够返回对第一个类的对象的引用。通过写一个继承自第一个类的匿名内部类，来创建一个返回对象。

10.6.1 再访工厂方法

看看在使用匿名内部类时，interfaces/Factories.java示例变得多么美妙呀：

```
//: innerclasses/Factories.java
import static net.mindview.util.Print.*;

interface Service {
  void method1();
  void method2();
}
```

360

361

```
interface ServiceFactory {
  Service getService();
}
class Implementation1 implements Service {
  private Implementation1() {}
  public void method1() {print("Implementation1 method1");}
  public void method2() {print("Implementation1 method2");}
  public static ServiceFactory factory =
    new ServiceFactory() {
      public Service getService() {
        return new Implementation1();
      }
    };
}

class Implementation2 implements Service {
  private Implementation2() {}
  public void method1() {print("Implementation2 method1");}
  public void method2() {print("Implementation2 method2");}
  public static ServiceFactory factory =
    new ServiceFactory() {
      public Service getService() {
        return new Implementation2();
      }
    };
}

public class Factories {
  public static void serviceConsumer(ServiceFactory fact) {
    Service s = fact.getService();
    s.method1();
    s.method2();
  }
  public static void main(String[] args) {
    serviceConsumer(Implementation1.factory);
    // Implementations are completely interchangeable:
    serviceConsumer(Implementation2.factory);
  }
} /* Output:
Implementation1 method1
Implementation1 method2
Implementation2 method1
Implementation2 method2
*///:~
```

现在用于**Implementation1**和**Implementation2**的构造器都可以是**private**的，并且没有任何必要去创建作为工厂的具名类。另外，你经常只需要单一的工厂对象，因此在本例中它被创建为**Service**实现中的一个**static**域。这样所产生语法也更具有实际意义。

362

interfaces/Games.java示例也可以通过使用匿名内部类来改进：

```
//: innerclasses/Games.java
// Using anonymous inner classes with the Game framework.
import static net.mindview.util.Print.*;

interface Game { boolean move(); }
interface GameFactory { Game getGame(); }

class Checkers implements Game {
  private Checkers() {}
  private int moves = 0;
  private static final int MOVES = 3;
  public boolean move() {
    print("Checkers move " + moves);
    return ++moves != MOVES;
  }
  public static GameFactory factory = new GameFactory() {
```

```
    public Game getGame() { return new Checkers(); }
  };
}

class Chess implements Game {
  private Chess() {}
  private int moves = 0;
  private static final int MOVES = 4;
  public boolean move() {
    print("Chess move " + moves);
    return ++moves != MOVES;
  }
  public static GameFactory factory = new GameFactory() {
    public Game getGame() { return new Chess(); }
  };
}

public class Games {
  public static void playGame(GameFactory factory) {
    Game s = factory.getGame();
    while(s.move())
      ;
  }
  public static void main(String[] args) {
    playGame(Checkers.factory);
    playGame(Chess.factory);
  }
} /* Output:
Checkers move 0
Checkers move 1
Checkers move 2
Chess move 0
Chess move 1
Chess move 2
Chess move 3
*///:~
```

363

请记住在第9章最后给出的建议：优先使用类而不是接口。如果你的设计中需要某个接口，你必须了解它。否则，不到迫不得已，不要将其放到你的设计中。

练习16：(1) 修改第9章中练习18的解决方案，让它使用匿名内部类。

练习17：(1) 修改第9章中练习19的解决方案，让它使用匿名内部类。

10.7 嵌套类

如果不需要内部类对象与其外围类对象之间有联系，那么可以将内部类声明为**static**。这通常称为*嵌套类*[⊖]。想要理解**static**应用于内部类时的含义，就必须记住，普通的内部类对象隐式地保存了一个引用，指向创建它的外围类对象。然而，当内部类是**static**的时，就不是这样了。嵌套类意味着：

1) 要创建嵌套类的对象，并不需要其外围类的对象。

2) 不能从嵌套类的对象中访问非静态的外围类对象。

364

嵌套类与普通的内部类还有一个区别。普通内部类的字段与方法，只能放在类的外部层次上，所以普通的内部类不能有**static**数据和**static**字段，也不能包含嵌套类。但是嵌套类可以包含所有这些东西：

```
//: innerclasses/Parcel11.java
// Nested classes (static inner classes).

public class Parcel11 {
```

⊖ 与C++嵌套类大致相似，只不过在C++中那些类不能访问私有成员，而在Java中可以访问。

```
  private static class ParcelContents implements Contents {
    private int i = 11;
    public int value() { return i; }
  }
  protected static class ParcelDestination
  implements Destination {
    private String label;
    private ParcelDestination(String whereTo) {
      label = whereTo;
    }
    public String readLabel() { return label; }
    // Nested classes can contain other static elements:
    public static void f() {}
    static int x = 10;
    static class AnotherLevel {
      public static void f() {}
      static int x = 10;
    }
  }
  public static Destination destination(String s) {
    return new ParcelDestination(s);
  }
  public static Contents contents() {
    return new ParcelContents();
  }
  public static void main(String[] args) {
    Contents c = contents();
    Destination d = destination("Tasmania");
  }
} ///:~
```

365

在**main()**中，没有任何**Parcel11**的对象是必需的；而是使用选取**static**成员的普通语法来调用方法——这些方法返回对**Contents**和**Destination**的引用。

就像你在本章前面看到的那样，在一个普通的（非**static**）内部类中，通过一个特殊的**this**引用可以链接到其外围类对象。嵌套类就没有这个特殊的**this**引用，这使得它类似于一个**static**方法。

练习18：(1) 创建一个包含嵌套类的类。在**main()**中创建其内部类的实例。

练习19：(2) 创建一个包含了内部类的类，而此内部类又包含有内部类。使用嵌套类重复这个过程。注意编译器生成的**.class**文件的名字。

10.7.1 接口内部的类

正常情况下，不能在接口内部放置任何代码，但嵌套类可以作为接口的一部分。你放到接口中的任何类都自动地是**public**和**static**的。因为类是**static**的，只是将嵌套类置于接口的命名空间内，这并不违反接口的规则。你甚至可以在内部类中实现其外围接口，就像下面这样：

```
//: innerclasses/ClassInInterface.java
// {main: ClassInInterface$Test}

public interface ClassInInterface {
  void howdy();
  class Test implements ClassInInterface {
    public void howdy() {
      System.out.println("Howdy!");
    }
    public static void main(String[] args) {
      new Test().howdy();
    }
  }
} /* Output:
Howdy!
*///:~
```

366

如果你想要创建某些公共代码，使得它们可以被某个接口的所有不同实现所共用，那么使用接口内部的嵌套类会显得很方便。

我曾在本书中建议过，在每个类中都写一个**main()**方法，用来测试这个类。这样做有一个缺点，那就是必须带着那些已编译过的额外代码。如果这对你是个麻烦，那就可以使用嵌套类来放置测试代码。

```
//: innerclasses/TestBed.java
// Putting test code in a nested class.
// {main: TestBed$Tester}

public class TestBed {
  public void f() { System.out.println("f()"); }
  public static class Tester {
    public static void main(String[] args) {
      TestBed t = new TestBed();
      t.f();
    }
  }
} /* Output:
f()
*///:~
```

这生成了一个独立的类**TestBed$Tester**（要运行这个程序，执行**java TestBed$Tester**即可，在**Unix/Linux**系统中必须转义**$**）。可以使用这个类来做测试，但是不必在发布的产品中包含它，在将产品打包前可以简单地删除**TestBed$Tester.class**。

练习20：(1) 创建一个包含嵌套类的接口，实现此接口并创建嵌套类的实例。

练习21：(2) 创建一个包含嵌套类的接口，该嵌套类中有一个**static**方法，它将调用接口中的方法并显示结果。实现这个接口，并将这个实现的一个实例传递给这个方法。 367

10.7.2 从多层嵌套类中访问外部类的成员

一个内部类被嵌套多少层并不重要[⊖]——它能透明地访问所有它所嵌入的外围类的所有成员，如下所示：

```
//: innerclasses/MultiNestingAccess.java
// Nested classes can access all members of all
// levels of the classes they are nested within.

class MNA {
  private void f() {}
  class A {
    private void g() {}
    public class B {
      void h() {
        g();
        f();
      }
    }
  }
}

public class MultiNestingAccess {
  public static void main(String[] args) {
    MNA mna = new MNA();
    MNA.A mnaa = mna.new A();
    MNA.A.B mnaab = mnaa.new B();
    mnaab.h();
  }
} ///:~
```

⊖ 再次感谢Martin Danner。

可以看到在MNA.A.B中，调用方法g()和f()不需要任何条件（即使它们被定义为private）。这个例子同时展示了如何从不同的类里创建多层嵌套的内部类对象的基本语法。".new"语法能产生正确的作用域，所以不必在调用构造器时限定类名。

368

10.8 为什么需要内部类

至此，我们已经看到了许多描述内部类的语法和语义，但是这并不能回答"为什么需要内部类"这个问题。那么，Sun公司为什么会如此费心地增加这项基本的语言特性呢？

一般说来，内部类继承自某个类或实现某个接口，内部类的代码操作创建它的外围类的对象。所以可以认为内部类提供了某种进入其外围类的窗口。

内部类必须要回答的一个问题是：如果只是需要一个对接口的引用，为什么不通过外围类实现那个接口呢？答案是："如果这能满足需求，那么就应该这样做。"那么内部类实现一个接口与外围类实现这个接口有什么区别呢？答案是：后者不是总能享用到接口带来的方便，有时需要用到接口的实现。所以，使用内部类最吸引人的原因是：

每个内部类都能独立地继承自一个（接口的）实现，所以无论外围类是否已经继承了某个（接口的）实现，对于内部类都没有影响。

如果没有内部类提供的、可以继承多个具体的或抽象的类的能力，一些设计与编程问题就很难解决。从这个角度看，内部类使得多重继承的解决方案变得完整。接口解决了部分问题，而内部类有效地实现了"多重继承"。也就是说，内部类允许继承多个非接口类型（译注：类或抽象类）。

为了看到更多的细节，让我们考虑这样一种情形：即必须在一个类中以某种方式实现两个接口。由于接口的灵活性，你有两种选择：使用单一类，或者使用内部类：

369

```
//: innerclasses/MultiInterfaces.java
// Two ways that a class can implement multiple interfaces.
package innerclasses;

interface A {}
interface B {}

class X implements A, B {}

class Y implements A {
  B makeB() {
    // Anonymous inner class:
    return new B() {};
  }
}

public class MultiInterfaces {
  static void takesA(A a) {}
  static void takesB(B b) {}
  public static void main(String[] args) {
    X x = new X();
    Y y = new Y();
    takesA(x);
    takesA(y);
    takesB(x);
    takesB(y.makeB());
  }
} ///:~
```

当然，这里假设在两种方式下的代码结构都确实有逻辑意义。然而遇到问题的时候，通常问题本身就能给出某些指引，告诉你是应该使用单一类，还是使用内部类。但如果没有任何其

他限制，从实现的观点来看，前面的例子并没有什么区别，它们都能正常运作。

如果拥有的是**抽象**的类或具体的类，而不是接口，那就只能使用内部类才能实现多重继承。

```
//: innerclasses/MultiImplementation.java
// With concrete or abstract classes, inner
// classes are the only way to produce the effect
// of "multiple implementation inheritance."
package innerclasses;

class D {}
abstract class E {}
class Z extends D {
  E makeE() { return new E() {}; }
}

public class MultiImplementation {
  static void takesD(D d) {}
  static void takesE(E e) {}
  public static void main(String[] args) {
    Z z = new Z();
    takesD(z);
    takesE(z.makeE());
  }
} ///:~
```

370

如果不需要解决“多重继承”的问题，那么自然可以用别的方式编码，而不需要使用内部类。但如果使用内部类，还可以获得其他一些特性：

1) 内部类可以有多个实例，每个实例都有自己的状态信息，并且与其外围类对象的信息相互独立。

2) 在单个外围类中，可以让多个内部类以不同的方式实现同一个接口，或继承同一个类。稍后就会展示一个这样的例子。

3) 创建内部类对象的时刻并不依赖于外围类对象的创建。

4) 内部类并没有令人迷惑的“is-a”关系；它就是一个独立的实体。

举个例子，如果**Sequence.java**不使用内部类，就必须声明“**Sequence**是一个**Selector**”，对于某个特定的**Sequence**只能有一个**Selector**。然而使用内部类很容易就能拥有另一个方法**reverseSelector()**，用它来生成一个反方向遍历序列的**Selector**。只有内部类才有这种灵活性。

练习22：(2) 实现**Sequence.java**中的**reverseSelector()**方法。

练习23：(4) 创建一个接口**U**，它包含三个方法。创建第一个类**A**，它包含一个方法，在此方法中通过创建一个匿名内部类，来生成指向**U**的引用。创建第二个类**B**，它包含一个由**U**构成的数组。**B**应该有几个方法，第一个方法可以接受对**U**的引用并存储到数组中；第二方法将数组中的引用设为**null**；第三个方法遍历此数组，并在**U**中调用这些方法。在**main()**中，创建一组**A**的对象和一个**B**的对象。用那些**A**类对象所产生的**U**类型的引用填充**B**对象的数组。使用**B**回调所有**A**的对象。再从**B**中移除某些**U**的引用。

371

10.8.1 闭包与回调

闭包（closure）是一个可调用的对象，它记录了一些信息，这些信息来自于创建它的作用域。通过这个定义，可以看出内部类是面向对象的闭包，因为它不仅包含外围类对象（创建内部类的作用域）的信息，还自动拥有一个指向此外围类对象的引用，在此作用域内，内部类有权操作所有的成员，包括**private**成员。

Java最引人争议的问题之一就是，人们认为Java应该包含某种类似指针的机制，以允许*回调*（callback）。通过回调，对象能够携带一些信息，这些信息允许它在稍后的某个时刻调用初始的

对象。稍后将会看到这是一个非常有用的概念。如果回调是通过指针实现的，那么就只能寄希望于程序员不会误用该指针。然而，读者应该已经了解到，Java更小心仔细，所以没有在语言中包括指针。

通过内部类提供闭包的功能是优良的解决方案，它比指针更灵活、更安全。见下例：

```
//: innerclasses/Callbacks.java
// Using inner classes for callbacks
package innerclasses;
import static net.mindview.util.Print.*;

interface Incrementable {
  void increment();
}

// Very simple to just implement the interface:
class Callee1 implements Incrementable {
  private int i = 0;
  public void increment() {
    i++;
    print(i);
  }
}

class MyIncrement {
  public void increment() { print("Other operation"); }
  static void f(MyIncrement mi) { mi.increment(); }
}

// If your class must implement increment() in
// some other way, you must use an inner class:
class Callee2 extends MyIncrement {
  private int i = 0;
  public void increment() {
    super.increment();
    i++;
    print(i);
  }
  private class Closure implements Incrementable {
    public void increment() {
      // Specify outer-class method, otherwise
      // you'd get an infinite recursion:
      Callee2.this.increment();
    }
  }
  Incrementable getCallbackReference() {
    return new Closure();
  }
}

class Caller {
  private Incrementable callbackReference;
  Caller(Incrementable cbh) { callbackReference = cbh; }
  void go() { callbackReference.increment(); }
}

public class Callbacks {
  public static void main(String[] args) {
    Callee1 c1 = new Callee1();
    Callee2 c2 = new Callee2();
    MyIncrement.f(c2);
    Caller caller1 = new Caller(c1);
    Caller caller2 = new Caller(c2.getCallbackReference());
    caller1.go();
    caller1.go();
    caller2.go();
```

372

373

```
    caller2.go();
  }
} /* Output:
Other operation
1
1
2
Other operation
2
Other operation
3
*///:~
```

这个例子进一步展示了外围类实现一个接口与内部类实现此接口之间的区别。就代码而言，**Callee1**是简单的解决方式。**Callee2**继承自**MyIncrement**，后者已经有了一个不同的**increment()**方法，并且与**Incrementable**接口期望的**increment()**方法完全不相关。所以如果**Callee2**继承了**MyIncrement**，就不能为了**Incrementable**的用途而覆盖**increment()**方法，于是只能使用内部类独立地实现**Incrementable**。还要注意，当创建了一个内部类时，并没有在外围类的接口中添加东西，也没有修改外围类的接口。

注意，在**Callee2**中除了**getCallbackReference()**以外，其他成员都是**private**的。要想建立与外部世界的任何连接，**interface Incrementable**都是必需的。在这里可以看到，**interface**是如何允许接口与接口的实现完全独立的。

内部类**Closure**实现了**Incrementable**，以提供一个返回**Callee2**的“钩子”（hook）——而且是一个安全的钩子。无论谁获得此**Incrementable**的引用，都只能调用**increment()**，除此之外没有其他功能（不像指针那样，允许你做很多事情）。

Caller的构造器需要一个**Incrementable**的引用作为参数（虽然可以在任意时刻捕获回调引用），然后在以后的某个时刻，**Caller**对象可以使用此引用回调**Callee**类。

回调的价值在于它的灵活性——可以在运行时动态地决定需要调用什么方法。这样做的好处在第22章可以看得更明显，在那里实现GUI功能的时候，到处都用到了回调。 374

10.8.2 内部类与控制框架

在将要介绍的*控制框架*（control framework）中，可以看到更多使用内部类的具体例子。

应用程序框架（application framework）就是被设计用以解决某类特定问题的一个类或一组类。要运用某个应用程序框架，通常是继承一个或多个类，并覆盖某些方法。在覆盖后的方法中，编写代码定制应用程序框架提供的通用解决方案，以解决你的特定问题（这是设计模式中*模板方法*的一个例子（参考www.MindVeiw.net上的《Thinking in Patterns (with Java)》）。模板方法包含算法的基本结构，并且会调用一个或多个可覆盖的方法，以完成算法的动作。设计模式总是将变化的事物与保持不变的事物分离开，在这个模式中，模板方法是保持不变的事物，而可覆盖的方法就是变化的事物。

控制框架是一类特殊的应用程序框架，它用来解决响应事件的需求。主要用来响应事件的系统被称作*事件驱动系统*。应用程序设计中常见的问题之一是图形用户接口（GUI），它几乎完全是事件驱动的系统。在第22章将会看到，Java Swing库就是一个控制框架，它优雅地解决了GUI的问题，并使用了大量的内部类。

要理解内部类是如何允许简单的创建过程以及如何使用控制框架的，请考虑这样一个控制框架，它的工作就是在事件“就绪”的时候执行事件。虽然“就绪”可以指任何事，但在本例中是指基于时间触发的事件。接下来的问题就是，对于要控制什么，控制框架并不包含任何具体的信息。那些信息是在实现算法的**action()**部分时，通过继承来提供的。

首先，接口描述了要控制的事件。因为其默认的行为是基于时间去执行控制，所以使用抽象类代替实际的接口。下面的例子包含了某些实现：

375

```
//: innerclasses/controller/Event.java
// The common methods for any control event.
package innerclasses.controller;
public abstract class Event {
  private long eventTime;
  protected final long delayTime;
  public Event(long delayTime) {
    this.delayTime = delayTime;
    start();
  }
  public void start() { // Allows restarting
    eventTime = System.nanoTime() + delayTime;
  }
  public boolean ready() {
    return System.nanoTime() >= eventTime;
  }
  public abstract void action();
} ///:~
```

当希望运行**Event**并随后调用**start()**时，那么构造器就会捕获（从对象创建的时刻开始的）时间，此时间是这样得来的：**start()**获取当前时间，然后加上一个延迟时间，这样生成触发事件的时间。**start()**是一个独立的方法，而没有包含在构造器内，因为这样就可以在事件运行以后重新启动计时器，也就是能够重复使用**Event**对象。例如，如果想要重复一个事件，只需简单地在**action()**中调用**start()**方法。

ready()告诉你何时可以运行**action()**方法了。当然，可以在导出类中覆盖**ready()**方法，使得**Event**能够基于时间以外的其他因素而触发。

下面的文件包含了一个用来管理并触发事件的实际控制框架。**Event**对象被保存在**List<Event>**类型（读作“Event的列表”）的容器对象中，容器会在第11章中详细介绍。目前读者只需要知道**add()**方法用来将一个**Object**添加到**List**的尾端，**size()**方法用来得到**List**中元素的个数，foreach语法用来连续获联**List**中的**Event**，**remove()**方法用来从**List**中移除指定的**Event**。

376

```
//: innerclasses/controller/Controller.java
// The reusable framework for control systems.
package innerclasses.controller;
import java.util.*;

public class Controller {
  // A class from java.util to hold Event objects:
  private List<Event> eventList = new ArrayList<Event>();
  public void addEvent(Event c) { eventList.add(c); }
  public void run() {
    while(eventList.size() > 0)
      // Make a copy so you're not modifying the list
      // while you're selecting the elements in it:
      for(Event e : new ArrayList<Event>(eventList))
        if(e.ready()) {
          System.out.println(e);
          e.action();
          eventList.remove(e);
        }
  }
} ///:~
```

run()方法循环遍历**eventList**，寻找就绪的（**ready()**）、要运行的**Event**对象。对找到的每一个就绪的（**ready()**）事件，使用对象的**toString()**打印其信息，调用其**action()**方法，然后从队列中移除此**Event**。

注意，在目前的设计中你并不知道Event到底做了什么。这正是此设计的关键所在，“使变化的事物与不变的事物相互分离”。用我的话说，“变化向量”就是各种不同的Event对象所具有的不同行为，而你通过创建不同的Event子类来表现不同的行为。

这正是内部类要做的事情，内部类允许：

1) 控制框架的完整实现是由单个的类创建的，从而使得实现的细节被封装了起来。内部类用来表示解决问题所必需的各种不同的action()。

2) 内部类能够很容易地访问外围类的任意成员，所以可以避免这种实现变得笨拙。如果没有这种能力，代码将变得令人讨厌，以至于你肯定会选择别的方法。

377

考虑此控制框架的一个特定实现，如控制温室的运作[⊖]：控制灯光、水、温度调节器的开关，以及响铃和重新启动系统，每个行为都是完全不同的。控制框架的设计使得分离这些不同的代码变得非常容易。使用内部类，可以在单一的类里面产生对同一个基类Event的多种导出版本。对于温室系统的每一种行为，都继承一个新的Event内部类，并在要实现的action()中编写控制代码。

作为典型的应用程序框架，GreenhouseControls类继承自Controller:

```
//: innerclasses/GreenhouseControls.java
// This produces a specific application of the
// control system, all in a single class. Inner
// classes allow you to encapsulate different
// functionality for each type of event.
import innerclasses.controller.*;

public class GreenhouseControls extends Controller {
  private boolean light = false;
  public class LightOn extends Event {
    public LightOn(long delayTime) { super(delayTime); }
    public void action() {
      // Put hardware control code here to
      // physically turn on the light.
      light = true;
    }
    public String toString() { return "Light is on"; }
  }
  public class LightOff extends Event {
    public LightOff(long delayTime) { super(delayTime); }
    public void action() {
      // Put hardware control code here to
      // physically turn off the light.
      light = false;
    }
    public String toString() { return "Light is off"; }
  }
  private boolean water = false;
  public class WaterOn extends Event {
    public WaterOn(long delayTime) { super(delayTime); }
    public void action() {
      // Put hardware control code here.
      water = true;
    }
    public String toString() {
      return "Greenhouse water is on";
    }
  }
  public class WaterOff extends Event {
```

378

⊖ 基于某种原因，我一直很乐意解决这个问题，该问题摘自我以前的书《C++ Inside & Out》，但是Java提供了更优雅的解决方案。

```
    public WaterOff(long delayTime) { super(delayTime); }
    public void action() {
      // Put hardware control code here.
      water = false;
    }
    public String toString() {
      return "Greenhouse water is off";
    }
  }
  private String thermostat = "Day";
  public class ThermostatNight extends Event {
    public ThermostatNight(long delayTime) {
      super(delayTime);
    }
    public void action() {
      // Put hardware control code here.
      thermostat = "Night";
    }
    public String toString() {
      return "Thermostat on night setting";
    }
  }
  public class ThermostatDay extends Event {
    public ThermostatDay(long delayTime) {
      super(delayTime);
    }
    public void action() {
      // Put hardware control code here.
      thermostat = "Day";
    }
    public String toString() {
      return "Thermostat on day setting";
    }
  }
  // An example of an action() that inserts a
  // new one of itself into the event list:
  public class Bell extends Event {
    public Bell(long delayTime) { super(delayTime); }
    public void action() {
      addEvent(new Bell(delayTime));
    }
    public String toString() { return "Bing!"; }
  }
  public class Restart extends Event {
    private Event[] eventList;
    public Restart(long delayTime, Event[] eventList) {
      super(delayTime);
      this.eventList = eventList;
      for(Event e : eventList)
        addEvent(e);
    }
    public void action() {
      for(Event e : eventList) {
        e.start(); // Rerun each event
        addEvent(e);
      }
      start(); // Rerun this Event
      addEvent(this);
    }
    public String toString() {
      return "Restarting system";
    }
  }
  public static class Terminate extends Event {
    public Terminate(long delayTime) { super(delayTime); }
    public void action() { System.exit(0); }
    public String toString() { return "Terminating";  }
```

379

```
  }
} ///:~
```

注意，**light**、**water** 和 **thermostat**都属于外围类**GreenhouseControls**，而这些内部类能够自由地访问那些字段，无需限定条件或特殊许可。而且，**action()**方法通常都涉及对某种硬件的控制。

380

大多数**Event**类看起来都很相似，但是**Bell**和**Restart**则比较特别。**Bell**控制响铃，然后在事件列表中增加一个**Bell**对象，于是过一会儿它可以再次响铃。读者可能注意到了内部类是多么像多重继承：**Bell**和**Restart**有**Event**的所有方法，并且似乎也拥有外围类**GreenhouseContrlos**的所有方法。

一个由**Event**对象组成的数组被递交给**Restart**，该数组要加到控制器上。由于**Restart()**也是一个**Event**对象，所以同样可以将**Restart**对象添加到**Restart.action()**中，以使系统能够有规律地重新启动自己。

下面的类通过创建一个**GreenhouseControls**对象，并添加各种不同的**Event**对象来配置该系统。这是命令设计模式的一个例子在eventList中的每一个被封装成对象的请求：

```
//: innerclasses/GreenhouseController.java
// Configure and execute the greenhouse system.
// {Args: 5000}
import innerclasses.controller.*;

public class GreenhouseController {
  public static void main(String[] args) {
    GreenhouseControls gc = new GreenhouseControls();
    // Instead of hard-wiring, you could parse
    // configuration information from a text file here:
    gc.addEvent(gc.new Bell(900));
    Event[] eventList = {
      gc.new ThermostatNight(0),
      gc.new LightOn(200),
      gc.new LightOff(400),
      gc.new WaterOn(600),
      gc.new WaterOff(800),
      gc.new ThermostatDay(1400)
    };
    gc.addEvent(gc.new Restart(2000, eventList));
    if(args.length == 1)
      gc.addEvent(
        new GreenhouseControls.Terminate(
          new Integer(args[0])));
    gc.run();
  }
} /* Output:
Bing!
Thermostat on night setting
Light is on
Light is off
Greenhouse water is on
Greenhouse water is off
Thermostat on day setting
Restarting system
Terminating
*///:~
```

381

这个类的作用是初始化系统，所以它添加了所有相应的事件。**Restart**事件反复运行，而且它每次都会将**eventList**加载到**GreenhouseControls**对象中。如果提供了命令行参数，系统会以它作为毫秒数，决定什么时候终止程序（这是测试程序时使用的）。当然，更灵活的方法是避免对事件进行硬编码，取而代之的是从文件中读取需要的事件（第12章的练习会要求读者照此方法

修改这个例子)。

这个例子应该使读者更了解内部类的价值了，特别是在控制框架中使用内部类的时候。在第18章中，读者将看到内部类如何优雅地描述图形用户界面的行为。到那时，读者应该就完全信服内部类的价值了。

练习24：(2) 在**GreenhouseControls.java**中增加一个**Event**内部类，用以打开、关闭风扇。配置**GreenhouseController.java**以使用这些新的**Event**对象。

练习25：(3) 在**GreenhouseControls.java**中继承**GreenhouseControls**，增加**Event**内部类，用以开启、关闭喷水机。写一个新版的**GreenhouseController.java**以使用这些新的**Event**对象。

10.9 内部类的继承

因为内部类的构造器必须连接到指向其外围类对象的引用，所以在继承内部类的时候，事情会变得有点复杂。问题在于，那个指向外围类对象的“秘密的”引用必须被初始化，而在导出类中不再存在可连接的默认对象。要解决这个问题，必须使用特殊的语法来明确说清它们之间的关联：

382

```
//: innerclasses/InheritInner.java
// Inheriting an inner class.

class WithInner {
  class Inner {}
}

public class InheritInner extends WithInner.Inner {
  //! InheritInner() {} // Won't compile
  InheritInner(WithInner wi) {
    wi.super();
  }
  public static void main(String[] args) {
    WithInner wi = new WithInner();
    InheritInner ii = new InheritInner(wi);
  }
} ///:~
```

可以看到，**InheritInner**只继承自内部类，而不是外围类。但是当要生成一个构造器时，默认的构造器并不算好，而且不能只是传递一个指向外围类对象的引用。此外，必须在构造器内使用如下语法：

```
enclosingClassReference.super();
```

这样才提供了必要的引用，然后程序才能编译通过。

练习26：(2) 创建一个包含内部类的类，此内部类有一个非默认的构造器（需要参数的构造器）。创建另一个也包含内部类的类，此内部类继承自第一个内部类。

10.10 内部类可以被覆盖吗

如果创建了一个内部类，然后继承其外围类并重新定义此内部类时，会发生什么呢？也就是说，内部类可以被覆盖吗？这看起来似乎是个很有用的思想，但是“覆盖”内部类就好像它是外围类的一个方法，其实并不起什么作用：

383

```
//: innerclasses/BigEgg.java
// An inner class cannot be overriden like a method.
import static net.mindview.util.Print.*;
class Egg {
  private Yolk y;
  protected class Yolk {
```

```
    public Yolk() { print("Egg.Yolk()"); }
  }
  public Egg() {
    print("New Egg()");
    y = new Yolk();
  }
}

public class BigEgg extends Egg {
  public class Yolk {
    public Yolk() { print("BigEgg.Yolk()"); }
  }
  public static void main(String[] args) {
    new BigEgg();
  }
} /* Output:
New Egg()
Egg.Yolk()
*///:~
```

默认的构造器是编译器自动生成的，这里是调用基类的默认构造器。你可能认为既然创建了**BigEgg**的对象，那么所使用的应该是“覆盖后”的**Yolk**版本，但从输出中可以看到实际情况并不是这样的。

这个例子说明，当继承了某个外围类的时候，内部类并没有发生什么特别神奇的变化。这两个内部类是完全独立的两个实体，各自在自己的命名空间内。当然，明确地继承某个内部类也是可以的：

```
//: innerclasses/BigEgg2.java
// Proper inheritance of an inner class.
import static net.mindview.util.Print.*;

class Egg2 {
  protected class Yolk {
    public Yolk() { print("Egg2.Yolk()"); }
    public void f() { print("Egg2.Yolk.f()");}
  }
  private Yolk y = new Yolk();
  public Egg2() { print("New Egg2()"); }
  public void insertYolk(Yolk yy) { y = yy; }
  public void g() { y.f(); }
}

public class BigEgg2 extends Egg2 {
  public class Yolk extends Egg2.Yolk {
    public Yolk() { print("BigEgg2.Yolk()"); }
    public void f() { print("BigEgg2.Yolk.f()"); }
  }
  public BigEgg2() { insertYolk(new Yolk()); }
  public static void main(String[] args) {
    Egg2 e2 = new BigEgg2();
    e2.g();
  }
} /* Output:
Egg2.Yolk()
New Egg2()
Egg2.Yolk()
BigEgg2.Yolk()
BigEgg2.Yolk.f()
*///:~
```

384

现在**BigEgg2.Yolk**通过**extends Egg2.Yolk**明确地继承了此内部类，并且覆盖了其中的方法。**insertYolk()**方法允许**BigEgg2**将它自己的**Yolk**对象向上转型为**Egg2**中的引用**y**。所以当**g()**调用**y.f()**时，覆盖后的新版的**f()**被执行。第二次调用**Egg2.Yolk()**，结果是**BigEgg2.Yolk**的构造器调用

了其基类的构造器。可以看到在调用g()的时候，新版的f()被调用了。

10.11 局部内部类

前面提到过，可以在代码块里创建内部类，典型的方式是在一个方法体的里面创建。局部内部类不能有访问说明符，因为它不是外围类的一部分；但是它可以访问当前代码块内的常量，以及此外围类的所有成员。下面的例子对局部内部类与匿名内部类的创建进行了比较。

```
//: innerclasses/LocalInnerClass.java
// Holds a sequence of Objects.
import static net.mindview.util.Print.*;

interface Counter {
  int next();
}

public class LocalInnerClass {
  private int count = 0;
  Counter getCounter(final String name) {
    // A local inner class:
    class LocalCounter implements Counter {
      public LocalCounter() {
        // Local inner class can have a constructor
        print("LocalCounter()");
      }
      public int next() {
        printnb(name); // Access local final
        return count++;
      }
    }
    return new LocalCounter();
  }
  // The same thing with an anonymous inner class:
  Counter getCounter2(final String name) {
    return new Counter() {
      // Anonymous inner class cannot have a named
      // constructor, only an instance initializer:
      {
        print("Counter()");
      }
      public int next() {
        printnb(name); // Access local final
        return count++;
      }
    };
  }
  public static void main(String[] args) {
    LocalInnerClass lic = new LocalInnerClass();
    Counter
      c1 = lic.getCounter("Local inner "),
      c2 = lic.getCounter2("Anonymous inner ");
    for(int i = 0; i < 5; i++)
      print(c1.next());
    for(int i = 0; i < 5; i++)
      print(c2.next());
  }
} /* Output:
LocalCounter()
Counter()
Local inner 0
Local inner 1
Local inner 2
Local inner 3
Local inner 4
```

385

386

```
Anonymous inner 5
Anonymous inner 6
Anonymous inner 7
Anonymous inner 8
Anonymous inner 9
*///:~
```

Counter返回的是序列中的下一个值。我们分别使用局部内部类和匿名内部类实现了这个功能，它们具有相同的行为和能力。既然局部内部类的名字在方法外是不可见的，那为什么我们仍然使用局部内部类而不是匿名内部类呢？唯一的理由是，我们需要一个已命名的构造器，或者需要重载构造器，而匿名内部类只能用于实例初始化。

所以使用局部内部类而不使用匿名内部类的另一个理由就是，需要不止一个该内部类的对象。

10.12 内部类标识符

由于每个类都会产生一个**.class**文件，其中包含了如何创建该类型的对象的全部信息（此信息产生一个“meta-class”，叫做**Class**对象），你可能猜到了，内部类也必须生成一个**.class**文件以包含它们的**Class**对象信息。这些类文件的命名有严格的规则：外围类的名字，加上“**$**”，再加上内部类的名字。例如，**LocalInnerClass.java**生成的**.class**文件包括：

```
Counter.class
LocalInnerClass$1.class
LocalInnerClass$1LocalCounter.class
LocalInnerClass.class
```

如果内部类是匿名的，编译器会简单地产生一个数字作为其标识符。如果内部类是嵌套在别的内部类之中，只需直接将它们的名字加在其外围类标识符与“**$**”的后面。 387

虽然这种命名格式简单而直接，但它还是很健壮的，足以应对绝大多数情况[⊖]。因为这是Java的标准命名方式，所以产生的文件自动都是平台无关的。（注意，为了保证你的内部类能起作用，Java编译器会尽可能地转换它们。）

10.13 总结

比起面向对象编程中其他的概念来，接口和内部类更深奥复杂；比如C++就没有这些。将两者结合起来，同样能够解决C++中的用多重继承所能解决的问题。然而，多重继承在C++中被证明是相当难以使用的，相比较而言，Java的接口和内部类就容易理解多了。

虽然这些特性本身是相当直观的，但是就像多态机制一样，这些特性的使用应该是设计阶段考虑的问题。随着时间的推移，读者将能够更好地识别什么情况下应该使用接口，什么情况使用内部类，或者两者同时使用。但此时，读者至少应该已经完全理解了它们的语法和语义。当见到这些语言特性实际应用时，就最终理解它们了。

所选习题的答案都可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，读者可以从www.MindView.net购买此文档。 388

⊖ 而另一方面，对于Unix shell而言，“$”是一个元字符，所以在列出**.class**文件的时候，有时会有问题。这对于基于Unix的Sun公司而言，真是有点奇怪。我猜这是因为他们没有考虑这个问题，他们认为你自然是应该专注于源码文件的。